1 Cycle-slip detection and repair

The uncombined filter (the observation-equations section of the Foundations document) forms no linear combination in its measurement update. The per-signal $P^i_{r,s}$ and $\Phi^i_{r,s}$ enter the uncombined code and phase observation equations directly. Cycle slips are screened by a set of *diagnostic* combinations. In each one the geometric range ρ^i_r, the clocks $c(\mathrm{d}t_r - \mathrm{d}t^i)$ and the troposphere T^i_r cancel. What remains varies between epochs only by measurement noise and by any integer discontinuity of the carrier.

The status of the detectors is as follows.

- Melbourne–Wübbena (`src/core/slip/MwSlipDetector.cpp`) is the primary detector and the only one armed by default.
- GRAPHIC (`src/core/slip/GraphicSlipDetector.cpp`) and the extra-wide-lane detector (`src/core/slip/EwlSlipDetector.cpp`), which needs a third frequency, are compiled and registered. Both are opt-in (Section 1.1), and no bundled config enables them.
- The ionosphere-free phase detector (`src/core/slip/IfPhaseSlipDetector.cpp`) is *compiled and armed on the network path*. It is instantiated per non-pivot station whenever `network.cycle_slip.if_phase_td` is set (Section 1.2.5). On the single-receiver user and kinematic path it stays staged, because the position-uncertainty gate it needs is unsafe there.
- The geometry-free phase detector (`src/core/slip/GfSlipDetector.cpp`) is *staged*. It is in the tree but commented out of the explicit CMake source list, unlike the IF detector beside it. Its algebra is recorded here for completeness.

The user and kinematic set is composed at run time from the config block `estimation.cycle_slip`. The network producer composes its own per-station set from a parallel `network.cycle_slip` block (Section 1.1).

Throughout this section we suppress the receiver index r (single-receiver PPP / PPP-RTK) and write $f_1 > f_2$ for the two carriers of a band pair, sorted by frequency so that $f_1 - f_2 > 0$ and $\lambda_1 < \lambda_2$ (`src/core/slip/MwSlipDetector.cpp`, the band-frequency sort). Carrier phase is read from RINEX in cycles and scaled by $\lambda_s = c/f_s$ before combining, so Φ^i_s below is always in metres, consistent with the phase observation equation. The operator $\delta\,\cdot$ denotes a *temporal* integer slip step: for example δN_s is the change in the ambiguity on signal s across the slip epoch. It is distinct from the between-satellite single difference $\Delta\cdot$ of the differencing section of the Foundations document, which also uses Δ.

1.1 Detector registry and composition

The set of detectors is configurable. Each detector implements the interface `ISlipDetector` (`include/slip/ISlipDetector.hpp`), which has one entry point per epoch

$$\texttt{detect}(\text{SlipEpochContext}) \;\longrightarrow\; \{\,\text{SlipEvent}\,\},$$

plus lifecycle hooks (`resetSatellite`, `resetSatelliteSignal`, `enableTracing`, `setCorrections`). A `SlipEvent` is the union of the legacy per-detector result structs. Its `granularity` field distinguishes two kinds of flag. A *per-signal* flag (GRAPHIC, GF, IF) names one carrier. A *band-difference* flag

(MW, EWL) reports a wide-lane integer $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$, which alone cannot assign the jump to one signal. Equation (15) resolves those two unknowns downstream. The inputs are bundled in `SlipEpochContext`, so every detector has the same signature. Only the IF detector uses the 3D position sigma and receiver ECEF it carries. On the network path, where that detector is armed, the position gate is always satisfied, because the estimator holds the station positions fixed.

`SlipDetectorRegistry::createEnabled` (`src/core/slip/SlipDetectorRegistry.cpp`) builds the set selected by `estimation.cycle_slip.{mw,graphic,ewl,gf,if}` in a fixed order. MW comes first. It is the primary detector and the only source of the wide-lane integers used by ambiguity resolution (the ambiguity-resolution document) and by the BSSD pivot logic (`src/core/filter/PivotManager.cpp`). The auxiliary detectors follow. MW defaults on and the auxiliaries default off, which keeps the historical MW-only behaviour.

A kind that is enabled in config but has no creator in the registry is skipped with a warning, so a config written for a later version still runs. Two kinds are skipped, for different reasons. The `.cpp` of `gf` is excluded from the build (Section 1.2.4). The `.cpp` of `if` *is* compiled, but the network path instantiates it directly (Section 1.2.5) and not through this registry, so `SlipDetectorRegistry` registers no `IonoFreePhase` creator.

`CycleSlipManager` drives the whole set through one loop. It reaches the MW detector's additional API (WL-lock queries, pivot-handover hints, per-satellite quality scoring) through a typed down-cast (`asMelbourneWubbena`), which keeps the base interface small.

The network producer does not use this registry for any detector. The `buildStation` lambda of `Application::networkSetup_` (`src/core/app/ApplicationNetworkSetup.cpp`) constructs each station's set directly from a *parallel* config block, `network.cycle_slip.{mw_detect, turbo_edit, if_phase_td}`. These keys are distinct from their `estimation.cycle_slip` namesakes, and all three default off. The two paths share the detector *classes*. They do not share the composition mechanism, the config block, or the response: the network refloats where the user path repairs (Section 1.3).

Composing by hand has one important consequence. With `network.cycle_slip.mw_detect` on and `network.cycle_slip.turbo_edit` off, the station receives an `MwSlipDetector` and a null `TurboEditResolver`. The null pointer is what directs the detection branch away from integer repair and to the refloat path.

`turbo_edit` (Section 1.4) is *not* a detector kind. It is a *coordination mode*. When the MW detector is enabled, it pairs the MW wide-lane integer with the geometry-free phase jump to recover per-signal integers. It is documented separately from the detectors below.

1.2 Test-statistic combinations

1.2.1 Melbourne–Wübbena (wide-lane)

The Melbourne–Wübbena (MW) observable is the wide-lane carrier phase minus the narrow-lane code. Introducing the wide- and narrow-lane wavelengths

$$\lambda_{\mathrm{WL}} := \frac{c}{f_1 - f_2}, \qquad \lambda_{\mathrm{NL}} := \frac{c}{f_1 + f_2}, \tag{1}$$

the engine forms (`MwSlipDetector::computeMwMeters`)

$$\boxed{\mathrm{MW}^i = \underbrace{\frac{f_1\,\Phi_1^i - f_2\,\Phi_2^i}{f_1 - f_2}}_{\text{wide-lane phase}} - \underbrace{\frac{f_1\,P_1^i + f_2\,P_2^i}{f_1 + f_2}}_{\text{narrow-lane code}}}. \tag{2}$$

The range-additive terms common to the uncombined code and phase observation equations, namely geometry, clocks and troposphere, cancel between the two bracketed combinations. The slant ionosphere cancels too. The wide-lane phase has the ionospheric coefficient $-(f_1\mu_1 - f_2\mu_2)/(f_1 - f_2)$ and the narrow-lane code has $+(f_1\mu_1 + f_2\mu_2)/(f_1 + f_2)$. With $\mu_s = f_1^2/f_s^2$ (the ionosphere-scaling relation) the two are equal, so (2) removes the first-order ionosphere exactly. What remains is nearly constant, equal to the wide-lane ambiguity plus hardware bias,

$$\mathrm{MW}^i \;\approx\; \lambda_{\mathrm{WL}}\, N^i_{\mathrm{WL}} \;+\; b^i_{\mathrm{MW}} \;+\; \varepsilon_{\mathrm{MW}}, \qquad N^i_{\mathrm{WL}} = N^i_1 - N^i_2, \tag{3}$$

so a wide-lane slip $\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2$ appears as a step of $\lambda_{\mathrm{WL}}\,\delta N_{\mathrm{WL}}$. For GPS L1/L2, $\lambda_{\mathrm{WL}} \approx 0.861\,\mathrm{m}$, and the dominant error is the narrow-lane *code* noise. With code noise $\sigma_P \sim 0.3\,\mathrm{m}$ the MW noise is $\sigma_{\mathrm{MW}} \approx \sigma_P/(\sqrt{2}\,\lambda_{\mathrm{WL}}) \sim 0.25$ wide-lane cycles, so the observable is limited by code noise. The integer-ready (BIA-corrected) variant subtracts the satellite MW bias b^i_{MW}, which has the same algebra as Equation (2) applied to the OSB phase and code biases of the uncombined code and phase observation equations.

The series $\mathrm{MW}^i(k)$ is tested against a robust adaptive gate. `estimation.cycle_slip.mw_detect_reference` selects the statistic.

The recommended choice is the first time difference $\mathrm{TD1}(k) = \mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$, gated against the median absolute deviation of its own history. Every bundled real-time and replay config sets it explicitly. The second time difference $\mathrm{TD2}(k) = \mathrm{MW}^i(k) - 2\,\mathrm{MW}^i(k-1) + \mathrm{MW}^i(k-2)$ is available (`mw_td_order`). It is insensitive to curvature, at a noise penalty of $\sqrt{6}$ against $\sqrt{2}$ for TD1. `MwSlipDetector::activeStatScale` holds the white-noise scaling of each statistic.

TD1 is *not* the loader default. A config that leaves `estimation.cycle_slip.mw_detect_reference` unset uses the legacy Hatch mean. It remains the default for backward compatibility and as an A/B arm, alongside the windowed local median (`hatch_mean`, `local_median`).

The threshold is derived from the data alone, and no σ from the stochastic model enters it. It combines a per-arc MAD, an EWMA-smoothed MAD floor across satellites, and a $1/\sin\theta$ elevation shape. For sparse or cold-start arcs the pool widens in steps: band pair, then constellation, then all satellites except GLONASS. The empirical construction is in `MwSlipDetector::computeAdaptiveThreshold` and is not reproduced here.

1.2.2 Extra-wide-lane (EWL)

For satellites with three or more frequencies the MW algebra (Equation (2)) is applied to a pair of *non-L1* carriers (`EwlSlipDetector::computeEwlMwMeters`; band digit ≥ 2). The two carriers are close in frequency, so $f_1 - f_2$ is small and the extra-wide-lane wavelength

$$\lambda_{\mathrm{EWL}} \;=\; \frac{c}{f_1 - f_2} \tag{4}$$

is very long: $\lambda_{\mathrm{EWL}} \approx 5.86\,\mathrm{m}$ for GPS L2/L5 and $\approx 9.77\,\mathrm{m}$ for Galileo E5a/E5b. Metre-level code noise is then $\lesssim 0.05$ EWL cycles. The EWL integer $N_{\mathrm{EWL}} = N_2 - N_5$ resolves in one epoch, and the size of a slip δN_{EWL} is determined almost without error. With the L1/L2 wide-lane (Equation (3)) this gives two independent band-difference constraints,

$$\delta N_{\mathrm{WL}} = \delta N_1 - \delta N_2, \qquad \delta N_{\mathrm{EWL}} = \delta N_2 - \delta N_5, \tag{5}$$

which leave the per-signal triple $(\delta N_1, \delta N_2, \delta N_5)$ with one degree of freedom [2].

1.2.3 GRAPHIC (single-frequency half-sum)

The GRAPHIC combination averages the code and phase on a *single* band (`GraphicSlipDetector::computeGraphic`):

$$\mathrm{GR}_s^i \;=\; \tfrac{1}{2}\big(P_s^i + \Phi_s^i\big), \tag{6}$$

with Φ_s^i in metres. The slant ionosphere enters as $+\mu_s I_{r,1}^i$ on the code and $-\mu_s I_{r,1}^i$ on the phase of the *same* signal (the uncombined code and phase observation equations), so the half-sum cancels it to first order. GRAPHIC is therefore ionosphere-free with one frequency, but it keeps half the code noise. The combination still contains the geometric range, the clocks, the troposphere, and the constant half-ambiguity $\frac{1}{2}\lambda_s N_s^i$. The between-epoch first difference

$$\mathrm{dGR}_s^i(k) \;=\; \mathrm{GR}_s^i(k) - \mathrm{GR}_s^i(k-1) \tag{7}$$

removes the constant ambiguity, and reduces range and clocks to the level of receiver motion and code noise. A slip of δN cycles on signal s produces a jump of $\frac{1}{2}\lambda_s\,\delta N$, and the integer estimate is $\delta N = \mathrm{round}\big(2\,\mathrm{dGR}/\lambda_s\big)$. With $\sigma_P \sim 0.3\,\mathrm{m}$ the noise after differencing is $\sim 0.3\,\mathrm{m}$, so the detector can resolve only a few L1 cycles. On quiet arcs the hard floor `kMinThresholdM` $= 0.5\,\mathrm{m}$ (≈ 5 L1 cycles) sets the threshold. GRAPHIC is therefore a coarse backstop and not a sub-cycle detector [1, 3].

1.2.4 Geometry-free phase (staged)

The geometry-free (GF) observable is the between-band phase difference

$$\mathrm{GF}^i \;=\; \Phi_1^i - \Phi_2^i \;\approx\; (\mu_2 - \mu_1)\, I_{r,1}^i \;+\; (\lambda_1 N_1^i - \lambda_2 N_2^i), \tag{8}$$

which removes geometry, clocks and troposphere exactly but keeps the *difference* of the two ionospheric delays. $I_{r,1}^i$ varies smoothly, so a second-order time difference

$$\mathrm{TD2}_{\mathrm{GF}}(k) \;=\; \mathrm{GF}^i(k) - 2\,\mathrm{GF}^i(k-1) + \mathrm{GF}^i(k-2) \tag{9}$$

cancels the ionospheric trend up to quadratic order over the sampling interval and leaves noise plus any slip (`GfSlipDetector::computeGfMeters` for Equation (8); the TD2 update is the test statistic for a level jump). The detector needs a high data rate ($\geq 1\,\mathrm{Hz}$). At a coarse rate the remaining ionospheric curvature is too large for (9). The detector is in the tree but excluded from the build.

The TD2 statistic (9) is still used elsewhere. `slip/IonoVariabilityMonitor` forms the same second difference per satellite on the two lowest RINEX bands (`selectPrimaryPair`, `computeGfMeters`), and it *is* compiled. It is a process-noise *monitor*, not a detector. Its per-satellite output is the clamped squared ratio of two MAD-derived values of σ for $\mathrm{TD2}_{\mathrm{GF}}$, one over a recent window and one over a baseline window,

$$m^i \;=\; \mathrm{clamp}\left(\left(\hat{\sigma}_{\mathrm{recent}}/\hat{\sigma}_{\mathrm{baseline}}\right)^2,\;\; 1,\;\; m_{\max}\right), \tag{10}$$

The ratio normalises itself. The slowly varying obliquity and curvature content of Equation (8) is present in *both* windows and cancels. A quiet sky gives $m^i \approx 1$ at any elevation, and the $1/\sin^2\theta$ obliquity scaling already in the ionospheric process noise is not counted twice. The monitor treats a cycle slip as an isolated single-epoch outlier, which the MAD is designed to *reject*. That is the opposite of a detection statistic, so the monitor is not a substitute for the staged detector.

Both arming keys default off. On the user path, `estimation.adaptive_iono_process_noise.enable` makes m^i scale the static ionospheric spectral density into a per-satellite zenith override, which `StateManager::setIonoProcessNoiseOverrides` consumes. On the network path, `network.adaptive_iono_q` (per station) makes it scale the deviation-channel innovation of the only time-linked slant-iono element the producer carries. The window lengths and the clamp $m_{\max}$ are empirical and stay in config.

1.2.5 Ionosphere-free phase (staged on the user path; armed on the network path)

The dual-frequency ionosphere-free (IF) phase combination uses the coefficients

$$c_1 = \frac{f_1^2}{f_1^2 - f_2^2}, \qquad c_2 = \frac{f_2^2}{f_1^2 - f_2^2}, \qquad c_1 - c_2 = 1, \tag{11}$$

giving $\Phi_{\mathrm{IF}}^i = c_1\Phi_1^i - c_2\Phi_2^i$ and the effective wavelength $\lambda_{\mathrm{IF}} = \left|c_1\lambda_1 - c_2\lambda_2\right|$ (`IfPhaseSlipDetector::computeIfWavelengthMeters`); for GPS L1/L2 $\lambda_{\mathrm{IF}} \approx 0.107\,\mathrm{m}$. The detector does not test the raw combination. It forms the IF observed-minus-computed (OMC) value and differences it twice. The first difference is between satellites, against a high-elevation pivot p_g. It removes the receiver clock, the inter-system offset $c\,\delta t_r^{(g)}$ and the satellite phase clock. The second is between epochs. It removes the slowly varying wet troposphere and the constant ambiguity:

$$\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}(k) = \left[\Phi_{\mathrm{IF}}^i(k) - \Phi_{\mathrm{IF}}^{p_g}(k)\right] - \left[\Phi_{\mathrm{IF}}^i(k-1) - \Phi_{\mathrm{IF}}^{p_g}(k-1)\right], \tag{12}$$

which is near zero for a clean arc and jumps by $\sim \delta N\,\lambda_{\mathrm{IF}}$ on a slip. The between-satellite step does not cancel the differential geometric error $(\boldsymbol{e}^i - \boldsymbol{e}^{p_g})^{\mathrm{T}}\delta\boldsymbol{r}$, so flagging must be gated by the filter's 3D position sigma. The gate requires a static site with a precise a priori position. The single-receiver kinematic path cannot guarantee that. There the detector is never instantiated, and the user toggle `estimation.enable_if_slip_detector` has no effect. The network estimator holds every station position *fixed*, so the gate is always satisfied, and the detector is armed there.

On the network path (the `buildStation` lambda of `Application::networkSetup_`, gated on `network.cycle_slip.if_phase_td`) an `IfPhaseSlipDetector` is instantiated per *non-pivot* station. The pivot's ambiguities are fixed by the gauge, so on the legacy datum path the pivot has nothing to refloat. Under the prioritised phase datum `network.phase_datum_prioritized` the detector is instantiated for every station, because the former pivot then has arcs that are not anchored and still need detection. The detector is built with `setDeferFlagging(true)`. It caches each per-epoch $\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g}$ of Equation (12) and does *not* apply a threshold locally. The driver (`networkSlipIfTdDriver` in `src/core/slip/NetworkSlipQc.cpp`) *pools* the deferred jumps across stations by signal pair and removes their median med. The median is the network common mode, a wobble of the frame or datum. It does not depend on elevation, so it is removed in raw metres. Each residual is then normalised to a zenith equivalent,

$$z \;=\; \left(\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\right)\sin\theta, \tag{13}$$

and the pooled robust scale $\hat{s} = 1.4826\,\mathrm{median}\,|z|$ is formed. At each satellite's threshold the scale is multiplied by $1/\sin\theta$ again, so *one* scale serves the whole sky. The map is the free function `ifElevMap` ($1/\sin\theta$, clamped at 5° so that it stays bounded near the horizon, which is already elevation-masked). The effective scale $\hat{s}_{\mathrm{eff}}$ is an empirical-Bayes blend of the pooled prior with the station pair's own EWMA scale. The shrinkage weight is $w = n/(n+\kappa)$, with n the number of clean samples of the station pair. A quiet reference station and a noisy station with modernised

signals therefore each get their own envelope, and the majority does not set one threshold for all. A satellite at elevation θ is flagged when

$$\left|\nabla\Delta\Phi_{\mathrm{IF}}^{i,p_g} - \mathrm{med}\right| \;>\; \max\!\left(\frac{k\,\hat{s}_{\mathrm{eff}}}{\sin\theta},\;\; c_{\mathrm{floor}}\,\lambda_{\mathrm{IF}}\right), \tag{14}$$

where k is a deliberately loose MAD multiplier. The floor is *proportional* to the $(1,1)$ IF jump $\lambda_{\mathrm{IF}} = c/(f_1+f_2) \approx 10.7\,\mathrm{cm}$ and is kept in raw metres, so a simultaneous one-cycle slip is caught at any elevation. Only the clean survivors are then folded into the station-pair EWMA, so the scale never absorbs the event it must catch. The five knobs (κ, k, c_{floor}, the minimum pool size and the EWMA weight) live under `network.cycle_slip.if_phase_td_*` and are snapshot once inside the driver.

A flag never repairs and never resets filter state in place. It restarts the detector's own arc unconditionally (arc hygiene), and its two estimator-side consequences are gated separately inside `refloat_pair`: the flagged pair enters the Card-4 invariant-carry slip funnel only when that carry is armed, and both band ambiguities are marked pending-reseed, the actual refloat, only once the phase-leveling carry is warm. On a cold or absent carry the flag is emitted as diagnostic telemetry and changes nothing, so a clean run log is not evidence that the arm acted; the run-end counters are. The sharp integer decision is left to a post-estimation phase validation that is not yet built, and the loose gate is intentional: this is a coarse first pass, not the discriminating solver of Section 1.4.

The pool cannot see a slip on the between-satellite reference itself: such a jump is common-mode across every satellite of that station's pool and is removed with med. The driver therefore folds in a second, independent flag source, the detector's own reference-satellite reset keys, parsed by `parseIfPairResetKey` and refloated through the same `refloat_pair`. Both bands of the pair must refloat because the IF combination cannot localise which one slipped. The reference-satellite election that produces those keys is itself gated by `IfPhaseSlipDetector::RefGateParams`, pushed only on the network path and armed by `network.cycle_slip.if_phase_td_ref_gate` (default off); the staged user/kinematic path keeps the legacy fixed gate.

1.3 Network slip QC: raw evidence and refloat over repair

The IF-phase driver of Section 1.2.5 is one of three arms of the network producer's slip layer. All three are free functions in `slip/NetworkSlipQc`, called in a fixed serial, station-ordered sequence from the per-epoch PASS-A slip seam (`Application::networkPassASlipScan_`). The other two are `networkSlipDrainTORepairs`, which drains the phase-leveling carry's integer repairs, and `networkSlipDrainMwRefloat`, which drains confident wide-lane integer jumps and the receiver's own LLI loss-of-lock bit. Their shared evidence is a fourth component, `slip/NetworkRawObservables`.

Shared evidence. `networkComputeRawObservables` forms, once per epoch per (station, satellite) on the two lowest distinct bands whose phase *and* matching code are both present, the same MW combination of Equation (2) and the same GF combination of Equation (8), plus the first band's phase-minus-code level and the per-band RINEX LLI byte. These are absolute *levels*, not differences. `networkQualifyRawObservables` then adjudicates each observable's step against the immediately adjacent processed epoch, that is $\mathrm{MW}^i(k) - \mathrm{MW}^i(k-1)$ and $\mathrm{GF}^i(k) - \mathrm{GF}^i(k-1)$, under the same threshold shape as Section 1.2.5: a multiple of an empirical scale, floored from below. The empirical term is a per-station robust EWMA of that station's *own* clean steps, built from the same $1.4826\,\mathrm{median}\,|\cdot|$ estimator (exposed for test as `networkRobustStepScale`); the floor is a *physical* integer bound kept in raw metres, a fraction of a wide-lane cycle λ_{WL} for MW, a few centimetres for GF. The gate therefore degrades to physics when the empirical term is cold or degenerate, never

to a stochastic-model σ, which is the same discipline as the MW adaptive threshold of Section 1.2.1. The ordering invariant also matches the IF-TD pool arm: each satellite is thresholded with the *pre-update* scale and only the clean survivors are folded into the EWMA, so the scale can never absorb the event it exists to catch. A step is measurable only against an *adjacent* predecessor; across a gap it is structurally unmeasurable, and the code is explicit that absence of evidence is neither evidence of a slip nor evidence of its absence. The multipliers and floor values live in `SlipQcParams`, not here.

Response doctrine. On the network path a detection *refloats*: it marks the ambiguity pending-reseed so the next pass re-estimates it from that epoch's data, and never applies an integer. This is a deliberate asymmetry against the user-path TurboEdit repair of Section 1.4, and the reason is the network's missing satellite phase biases: without them the network wide-lane integer is not trustworthy, so a wrong integer written into a carried value would bias the gauge, whereas a refloat costs only re-convergence. The one exception is `networkSlipDrainT0Repairs`, which shifts the phase-leveling carry by $\delta N_s\,\lambda_s$ *continuously*, variance unchanged and covariance-preserving, and only for an integer that is both confident (small rounding residual, Equation (20)) and of sane magnitude. A low-confidence detection is left untouched rather than repaired, refloated or dropped, because the float wide-lane's marginal repairs are measured to be predominantly false positives and acting on them discards a good carry.

Status. Every arm is default *off*: `network.cycle_slip.mw_detect` arms the MW/LLI drain, `network.cycle_slip.mw_lli_corroborate` the LLI qualification and `network.cycle_slip.gap_instrument` the arc-resumption instrument. The raw observables are computed at all only when some consumer asked for them (`SlipQcParams::anyConsumer`), and the adjudication above runs only when a *decision* card is armed; a run with the raw trace alone computes the levels and adjudicates nothing. The drains therefore guard every use of the verdicts on whether the evidence exists this epoch; without it the LLI arm stays decisive, because a gate that fired on missing data would be worse than the chatter it exists to fix. The qualification never *excludes* an LLI assertion: an assertion acts when a measured MW or GF step corroborates it, or when the station's own LLI behaviour is still judged credible by a two-state hysteresis on the EWMA of its LLI-set fraction (`networkLliCredibilityStep`). Only an uncorroborated assertion at a demonstrably non-credible station is downgraded, and it is then emitted as suppressed and counted rather than silently dropped.

The raw step evidence has a second consumer outside this chapter. With `network.carry_invariant_innov_corroborate` armed (default off), the Card-4 innovation gate on the carried double differences will not re-tap a chord whose arc the raw observables positively measured as continuous. That gate is fail-safe in one direction only: measured-and-flat suppresses, absence of evidence still re-taps, so the slip response is narrowed, never removed. The gate itself is estimation-side and is described in the Network mode document.

1.4 TurboEdit dual-frequency resolution

The TurboEdit resolver is a *pure* MW+GF joint integer solver (`src/core/slip/TurboEditResolver.cpp` and its header `include/slip/TurboEditResolver.hpp`). It owns the per-arc geometry-free jump history and the integer arithmetic, and returns per-signal integer repairs plus a reset-fallback set without mutating filter state; it is wired into both the post-processing `CycleSlipManager` and the real-time engine, and enabled by `estimation.cycle_slip.turbo_edit`.

That key is default *off*. The bundled real-time and SBF-replay configs turn it on; the internal PPP-RTK AR config leaves it off. The network producer has a separate key, `network.cycle_slip.turbo_edit`, and with that off but `network.cycle_slip.mw_detect` on the same MW detector runs as a pure detector and the network refloats rather than repairing (Section 1.3). Of the sub-toggles below, the soft-LLI bridge (`turbo_edit_bridge_lli`) is likewise default off, while the MW-only code-domain veto (`turbo_edit_mw_only_veto`) is default on.

1.4.1 The 2×2 system

For a dual-frequency phase pair (s_1, s_2) on satellite i, ordered higher-frequency first ($f_{s_1} > f_{s_2}$, hence $\lambda_{s_1} < \lambda_{s_2}$), the resolver pairs two scalar slip observables. The first is the MW wide-lane integer of Equation (3), taken *as input* from the MW detector and never recomputed (the empirically tuned MW path of Section 1.2.1 is the single source of δN_{WL}):

$$\delta N_{\mathrm{WL}} \;=\; \delta N_{s_1} - \delta N_{s_2} \qquad \text{(one equation, two unknowns).} \tag{15}$$

The second is the GF phase jump in metres. Writing the GF level as $\mathrm{GF}(t) = \lambda_{s_1} L_{s_1}(t) - \lambda_{s_2} L_{s_2}(t)$ (carrier in cycles), a simultaneous slip steps its ambiguity content by

$$\delta\mathrm{GF} \;=\; \lambda_{s_1}\,\delta N_{s_1} - \lambda_{s_2}\,\delta N_{s_2}, \tag{16}$$

estimated in-arc by the second-order time difference $\mathrm{TD2}(t) = \mathrm{GF}(t) - 2\,\mathrm{GF}(t-1) + \mathrm{GF}(t-2)$ of Equation (9), which cancels the ionospheric delay up to quadratic order so that at the slip epoch it equals the integer jump plus phase noise only. The TD2 chain requires three *consecutive* epochs; non-consecutive history forces a chain re-seed (Section 1.4.4).

Stacking Equations (15) and (16) gives the linear map from the integer slip pair to the two observed jumps:

$$\boxed{\begin{pmatrix} \delta N_{\mathrm{WL}} \\ \delta\mathrm{GF} \end{pmatrix} \;=\; \underbrace{\begin{pmatrix} 1 & -1 \\ \lambda_{s_1} & -\lambda_{s_2} \end{pmatrix}}_{\boldsymbol{A}} \begin{pmatrix} \delta N_{s_1} \\ \delta N_{s_2} \end{pmatrix}.} \tag{17}$$

The determinant $\det \boldsymbol{A} = \lambda_{s_1} - \lambda_{s_2}$ is non-zero for any genuine cross-band pair: it is the cm-scale quantity `denom` of Equation (19), $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, *not* the $\sim 0.86\,\mathrm{m}$ wide-lane wavelength, so the system is always invertible:

$$\boldsymbol{A}^{-1} = \frac{1}{\lambda_{s_1} - \lambda_{s_2}} \begin{pmatrix} -\lambda_{s_2} & 1 \\ -\lambda_{s_1} & 1 \end{pmatrix}. \tag{18}$$

The implementation does not invert the full matrix; it solves the second row for δN_{s_2} and back-substitutes the first (`TurboEditResolver.cpp`):

$$\delta N_{s_2} \;=\; \frac{\delta\mathrm{GF} - \lambda_{s_1}\,\delta N_{\mathrm{WL}}}{\lambda_{s_1} - \lambda_{s_2}}, \qquad \delta N_{s_1} \;=\; \delta N_{s_2} + \delta N_{\mathrm{WL}}, \tag{19}$$

with $\delta\mathrm{GF}$ instantiated either by the in-arc TD2 or, across a data gap, by the bridged jump of Section 1.4.4. The denominator $\lambda_{s_1} - \lambda_{s_2}$ (the variable `denom` in the source) equals $\approx -0.0539\,\mathrm{m}$ for GPS L1/L2, so the amplification factor on the wide-lane integer is $\lambda_{s_1}/(\lambda_{s_1} - \lambda_{s_2}) \approx -3.53$: the GF equation must localise δN_{s_2} to well within ± 0.5 cycle despite that lever, which is what the split-capability gate below enforces.

1.4.2 Integer rounding and validation

The float δN_{s_2} from Equation (19) is rounded and its residual recorded:

$$d_2 = \mathrm{round}\big(\delta N_{s_2}\big), \quad f = \big|\delta N_{s_2} - d_2\big|, \quad d_1 = d_2 + \delta N_{\mathrm{WL}}. \tag{20}$$

A candidate is *accepted* as a repair only if the rounding is clean ($f \le 0.30$, the constant `kMaxFractional`; a $1 - 0.30 = 0.70$ cycle margin to the nearest wrong integer), the magnitudes are sane ($|d_1|, |d_2| \le$ `kMaxSlipCycles`), and the pair is non-trivial ($d_1 \neq 0 \vee d_2 \neq 0$). A non-trivial accepted candidate yields `action="repair"`; $(d_1, d_2) = (0, 0)$ is resolved as no-slip and the arc keeps rolling; otherwise the satellite is reset.

Before the joint solve is *trusted*, the GF evidence must be sharp enough to discriminate the integer lattice. The float δN_{s_2} carries $\sigma_{\mathrm{TD2}}/|\lambda_{s_1} - \lambda_{s_2}|$ cycles of noise, so the solve is permitted only when

$$\sigma_{\mathrm{TD2}} \;\le\; \texttt{turbo_edit_split_max_sigma} \cdot \big|\lambda_{s_1} - \lambda_{s_2}\big| \qquad (\texttt{splitCapable}), \tag{21}$$

with σ_{TD2} the arc's own MAD-derived σ when mature, else the cross-satellite tier floor, else NaN (NaN fails by design, since "no estimate" is not evidence of sharpness). The default `estimation.cycle_slip.turbo_edit_split_max_sigma` is 0.2 cycles. This gate is the single mechanism making the resolver rate-adaptive: millimetric 1 s noise passes trivially, while coarse-rate (30 s) ionospheric residual passes only while the arc is quiet. On an MW-flagged pair failing the gate the resolver falls back to a satellite reset; a GF-only exceedance on a split-incapable arc is *ignored*, not reset. When $\delta N_{\mathrm{WL}} = 0$ (an equal slip $\delta N_{s_1} = \delta N_{s_2}$, to which MW is blind) a GF-only flag fires on |TD2| exceeding an adaptive threshold $\tau = \max(k_\sigma \hat{\sigma}_{\mathrm{TD2}}, 0.04\,\mathrm{m})$, with k_σ the `turbo_edit_sigma_multiplier`; the floor sits just below $|\lambda_{s_1} - \lambda_{s_2}| \approx 0.054$ m so a ± 1-cycle equal slip remains detectable.

1.4.3 MW-only / code-event null line

The asymmetric case is MW flagged but the GF jump *affirmatively zero*. For a fixed $\delta N_{\mathrm{WL}} = k$, every integer pair $(\delta N_{s_1}, \delta N_{s_1} - k)$ predicts, via Equation (16), a GF jump

$$\delta\mathrm{GF} \;=\; \lambda_{s_2} k - (\lambda_{s_2} - \lambda_{s_1})\, \delta N_{s_1}, \tag{22}$$

whose zero crossing sits at $\delta N_{s_1} = \lambda_{s_2} k/(\lambda_{s_2} - \lambda_{s_1}) \approx 4.53\,k$ for GPS L1/L2 ($\approx 3.95\,k$ for L1/L5). Every small pair $|\delta N_{s_1}| \le 3$ then predicts $|\delta\mathrm{GF}| \ge 8$ cm, orders above phase noise, so a flat GF leaves only *null-line* pairs of $(4, 3)$ / $(9, 7)$-class magnitude: implausible multi-cycle simultaneous slips on both signals with no loss of lock. The Occam-simpler explanation is a narrow-lane *code* excursion (multipath, NLOS, DLL re-lock), since $\mathrm{MW} = \Phi_{\mathrm{WL}} - P_{\mathrm{NL}}$ and one wide-lane cycle ≈ 0.86 m of narrow-lane code.

Accordingly the pair is classified a code-domain artifact, and the caller *skips* the phase reset, when the GF jump is affirmatively zero within a discrimination-bounded gate,

$$|\mathrm{TD2}| \le \underbrace{\texttt{kCodeEventZeroSigma} \cdot \hat{\sigma}_{\mathrm{MAD}}}_{\text{zero gate}} \quad \wedge \quad \text{zero gate} \le \underbrace{\texttt{kVetoZeroGateMaxLambdaFrac} \cdot |\lambda_{s_1} - \lambda_{s_2}|}_{\text{discrimination cap}}, \tag{23}$$

with `kCodeEventZeroSigma` $= 3.5$ ($\approx 99.95\%$ two-sided) and `kVetoZeroGateMaxLambdaFrac` $= 0.5$. The cap is the rate-adaptive safeguard: a zero gate wider than half the smallest GF-visible quantum cannot distinguish stillness from a small slip, so the veto stands down and the MW mandate governs. The classification is further trust-gated: veto enabled, MW flagged, GF not exceeding, no post-slip cooldown, ionospherically stable constellation, no LLI on either signal, and MAD maturity ≥ 10

samples (stricter than detection's 5, because affirming stillness is a state-*preserving* decision). The implied null-line pair is computed for the trace only, never applied; `did_slip` stays false so the GF chain keeps rolling. A genuine null-line slip is not lost: it jumps that signal's Kalman phase innovation by $\geq |4\lambda_{s_1}| \approx 0.76\,\mathrm{m}$ this very epoch, far beyond any residual gate, so the residual-driven reset corrects it one epoch late. The behaviour is toggled by `estimation.cycle_slip.turbo_edit_mw_only_veto` (default on).

1.4.4 Gap-spanning GF bridge

The TD2 chain needs three consecutive epochs; any per-signal dropout or receiver gap re-seeds the chain, making the post-gap TD2 *blind* to a re-lock slip whose GF level jump lives *between* the two chains. When a chain break occurs within `estimation.cycle_slip.turbo_edit_max_gap_s`, the resolver stashes the pre-gap GF level $\mathrm{GF}_{\mathrm{ref}}$, its timestamp, the arc's EWMA GF rate $\dot{\mathrm{GF}}$ and the pre-gap spacing dt_0, and on return forms the drift-corrected across-gap jump

$$\boxed{\delta\mathrm{GF}_{\mathrm{gap}} \;=\; \mathrm{GF}_{\mathrm{now}} - \mathrm{GF}_{\mathrm{ref}} - \dot{\mathrm{GF}}\,\Delta t, \qquad \Delta t = t_{\mathrm{now}} - t_{\mathrm{ref}},} \tag{24}$$

which then plays the role of δGF in Equation (19). Here $\dot{\mathrm{GF}}$ is an EWMA ($\alpha = 0.1$) of the per-epoch GF rate, chosen so its noise does not amplify when projected over the gap. Because the un-cancelled ionospheric residual is random-walk over the gap, the detection threshold and the code-event zero gate are widened by the factor $w = \sqrt{\Delta t/dt_0}$ (for $\Delta t > dt_0$), and the split-capability σ becomes $\sigma_{\mathrm{bridge}} = \hat{\sigma}_{\mathrm{MAD}}\, w$; the zero-gate *discrimination cap* stays un-widened, since the integer lattice of Equation (17) does not stretch with the gap. The bridge cap default is 15 s in code, raised to 60 s in the bundled real-time and replay configs.

An MW-flagged pair across the gap is attributed three ways. (i) A *code event* (`action="code_event"`) when $\delta\mathrm{GF}_{\mathrm{gap}}$ is affirmatively zero within the widened gate, the same null-line logic as Equation (23), applied across the gap, preserving phase through a re-acquisition DLL transient. (ii) A *repair* or *moot* when the joint solve runs with $\delta\mathrm{GF} = \delta\mathrm{GF}_{\mathrm{gap}}$; per signal it is dispatched by observation-domain continuity, a signal observed at every epoch through the gap kept its ambiguity (its nonzero δN is a genuine *repair*), whereas a signal that missed an epoch was pruned-and-reseeded on return (its nonzero δN is *moot*, already absorbed, mandate suppressed). (iii) A *reset* when the across-gap σ is too coarse for the split, or the candidate set fits zero or two integers with no deferral.

Because the GF equation alone is degenerate along the null-line lattice of Equation (22) ($+0.735\,\mathrm{m}$ fits both $(0, -3)$ and $(9, 4)$ at millimetre level; only the integer k separates them), the bridged solve demands a *unique* surviving candidate after three guards: the fractional gate of Equation (20); a GF-separation guard requiring the candidate's own predicted jump to clear the zero gate, $|\lambda_{s_1} d_1 - \lambda_{s_2} d_2| >$ `zero_gate`, lest a marginal noise exceedance mint a multi-cycle repair from noise; and a sibling-pair zero affirmation, whereby a satellite's *other* GF arc that stayed chain-continuous and unflagged through the bridged interval affirms $\delta N = 0$ for both its signals and prunes any candidate demanding a nonzero integer on an affirmed signal. The wide-lane integer feeding the bridged solve is taken in trust order: the published MW integer k; failing that, at the flag epoch, $\mathrm{round}(f) \pm 3$ around the (NL-code-biased) instantaneous re-acquisition float; and on deferral epochs the settled across-gap MW Hatch mean. When the float wide-lane evidence is unsettled or the lattice is ambiguous (two or more acceptable integers), the solve is *deferred* rather than guessed (`estimation.cycle_slip.turbo_edit_wl_settle_max_s`), terminating in a later repair, reset or timeout-reset.

1.4.5 Atomic reconciliation

Per-satellite candidates are reconciled atomically: a cross-pair integer disagreement on a shared signal demotes the whole satellite to reset, and a satellite ending in the reset set has its code-event, moot, clean and pending classifications *withdrawn*, since contradictory evidence always resolves to the conservative outcome. Arc bookkeeping keeps the TD2 validity precondition honest: a non-consecutive update partially resets the chain but preserves the slowly-varying MAD calibration window, an absolute gap beyond the configured maximum wipes the arc, and an accepted slip re-seeds with a short cooldown to suppress the TD2 echo; only clean (non-exceeding, non-bridged) TD2 samples feed the MAD window.

References

[1] Simon Banville and Richard B. Langley. "Mitigating the impact of ionospheric cycle slips in GNSS observations". In: *Journal of Geodesy* 87.2 (2013), pp. 179–193. DOI: 10.1007/s00190-012-0604-1.

[2] Jianghui Geng and Yehuda Bock. "Triple-frequency GPS precise point positioning with rapid ambiguity resolution". In: *Journal of Geodesy* 87.5 (2013), pp. 449–460. DOI: 10.1007/s00190-013-0619-2.

[3] Thomas P. Yunck. "Coping with the atmosphere and ionosphere in precise satellite and ground positioning". In: *Environmental Effects on Spacecraft Positioning and Trajectories*. Geophysical Monograph Series 73 (1993), pp. 1–16. DOI: 10.1029/GM073p0001.